

Operating Systems - Unit 3

Process Synchronization

Race Conditions

A race condition occurs when several processes access and manipulate shared data concurrently and the outcome of the execution depends on the particular order in which the access takes place. To guard against race conditions, we need to ensure that only one process at a time can manipulate the shared variable, which requires that the processes be synchronized in some way. Race conditions are notoriously hard to reproduce because they depend on timing and scheduling decisions made by the kernel.

The Critical Section Problem

Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table or writing a file. A solution to the critical section problem must satisfy three requirements: mutual exclusion, progress and bounded waiting. Mutual exclusion means that if one process is executing in its critical section, then no other process can be executing in its critical section. Bounded waiting limits how many times other processes may enter their critical sections after a process has made a request.

Semaphores and Monitors

Semaphores

A semaphore is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait and signal. A counting semaphore can range over an unrestricted domain, while a binary semaphore can range only between zero and one and behaves like a mutex lock. Semaphores can be implemented without busy waiting by placing a blocked process in a waiting queue associated with the semaphore and waking it with a signal operation.

Monitors

A monitor is a high level abstraction that provides a convenient and effective mechanism for process synchronization. Only one process may be active within the monitor at a time, so the programmer does not need to code this synchronization constraint explicitly. Condition variables inside a monitor allow a process to wait until a particular condition becomes true, using the wait and signal operations on that condition.

Deadlocks

Necessary Conditions

A deadlock is a situation in which every process in a set is waiting for an event that can be caused only by another process in the set. Four conditions must hold simultaneously for a deadlock to occur: mutual exclusion, hold and wait, no preemption and circular wait. A resource allocation graph can be used to describe deadlocks precisely: if the graph contains no cycle, then no process in the system is deadlocked.

Deadlock Prevention

Deadlock prevention provides a set of methods to ensure that at least one of the necessary conditions cannot hold. For example, the hold and wait condition can be broken by requiring each process to request and be allocated all its resources before it begins execution. The circular wait condition can be broken by imposing a total ordering of all resource types and requiring that each process requests resources in increasing order of enumeration.

Deadlock Avoidance and the Banker's Algorithm

Deadlock avoidance requires that the operating system be given additional information in advance concerning which resources a process will request during its lifetime. A state is safe if the system can allocate resources to each process in some order and still avoid a deadlock. The Banker's algorithm uses the Available, Max, Allocation and Need matrices to decide whether granting a request leaves the system in a safe state; if it does not, the requesting process must wait.

Memory Management

Contiguous Allocation

In contiguous memory allocation, each process is contained in a single section of memory that is contiguous to the section containing the next process. The first fit, best fit and worst fit strategies are used to select a free hole from the set of available holes. Both first fit and best fit suffer from external fragmentation, where enough total memory space exists to satisfy a request but the available spaces are not contiguous; compaction is one solution.

Paging

Paging is a memory management scheme that permits the physical address space of a process to be noncontiguous. Physical memory is broken into fixed sized blocks called frames and logical memory into blocks of the same size called pages. Every address generated by the CPU is divided into a page number and a page offset; the page number is used as an index into a page table. A translation lookaside buffer caches recent page table entries to speed up address translation.

Segmentation

Segmentation is a memory management scheme that supports the programmer view of memory as a collection of variable sized segments such as the main program, functions, stacks and arrays. A logical address consists of a segment number and an offset. Each entry in the segment table has a segment base, which contains the starting physical address, and a segment limit, which specifies the length of the segment.

Virtual Memory

Demand Paging

Virtual memory is a technique that allows the execution of processes that are not completely in memory. With demand paging, pages are loaded only when they are demanded during program execution, so pages that are never accessed are never loaded into physical memory. When a process tries to access a page that is not in memory, a page fault trap occurs and the operating system brings the desired page into a free frame.

Page Replacement

If no frame is free, a page replacement algorithm selects a victim frame to write out. FIFO replacement evicts the oldest page and can suffer from Belady's anomaly, where more frames produce more page faults. The optimal algorithm replaces the page that will not be used for the longest period of time, and least recently used replacement approximates it by evicting the page that has not been used for the longest time. Thrashing happens when a process spends more time paging than executing.